1. ভূমিকা

এই রিপোর্টে আমরা **Backpropagation** অ্যালগরিদম ব্যবহার করে একটি নিউরাল নেটওয়ার্ক তৈরি ও বিশ্লেষণ করেছি, যা ইমেজ ক্লাসিফিকেশন সমস্যার সমাধানে ব্যবহৃত হয়। Backpropagation হলো একটি গুরুত্বপূর্ণ লার্নিং মেকানিজম, যার মাধ্যমে নিউরাল নেটওয়ার্ক তার পূর্বাভাসের ভুল (error) বিশ্লেষণ করে এবং ধাপে ধাপে তার ওজন (weights) আপডেট করে আরও ভালো ফলাফল দেয়।

এই ল্যাব রিপোর্টে আমাদের প্রধান লক্ষ্য ছিল একটি নিউরাল নেটওয়ার্ক তৈরি করা, যা **Fashion MNIST dataset** ব্যবহার করে বিভিন্ন ধরনের পোশাকের ছবি সঠিকভাবে শ্রেণীবদ্ধ (classify) করতে পারে। এই প্রক্রিয়ায় আমরা ডেটা লোডিং, ডেটা ভিজুয়ালাইজেশন, ডেটা প্রিপ্রসেসিং, মডেল তৈরি, ট্রেনিং এবং মডেলের পারফরম্যান্স মূল্যায়ন করেছি।

এছাড়াও, আমরা মডেলের পূর্বাভাস (prediction) বিশ্লেষণ করেছি এবং কোথায় মডেল ভুল করছে তা পর্যবেক্ষণ করেছি, যা Backpropagation এর কার্যকারিতা বোঝার জন্য অত্যন্ত গুরুত্বপূর্ণ।

2. ডেটাসেট বর্ণনা

এই প্রজেক্টে আমরা **Fashion MNIST dataset** ব্যবহার করেছি, যা Zalando Research দ্বারা তৈরি একটি জনপ্রিয় ইমেজ ডেটাসেট। এটি মূলত MNIST dataset-এর একটি উন্নত সংস্করণ, যেখানে হাতে লেখা সংখ্যার পরিবর্তে বিভিন্ন ধরনের পোশাকের ছবি ব্যবহার করা হয়েছে।

ডেটাসেটটির বৈশিষ্ট্যগুলো নিম্নরূপ:

- মোট ইমেজ সংখ্যা: **70,000**
- ট্রেনিং ডেটা: **60,000** ইমেজ
- টেস্ট ডেটা: **10,000** ইমেজ
- প্রতিটি ইমেজের সাইজ: **28 × 28** পিক্সেল
- প্রতিটি ইমেজ গ্রেস্কেল (grayscale), অর্থাৎ শুধুমাত্র সাদা-কালো
- প্রতিটি ইমেজ একটি নির্দিষ্ট ক্লাসকে নির্দেশ করে

ডেটাসেটে মোট **10**টি ক্লাস রয়েছে, যেমন:

- T-shirt/top
- Trouser
- Pullover

- Dress
- Coat
- Sandal
- Shirt
- Sneaker
- Bag
- Ankle boot

ডেটাসেটটি Keras লাইব্রেরির মাধ্যমে সহজেই লোড করা যায়:

```
from keras.datasets import fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

এখানে:

- `train_images` এর shape: (60000, 28, 28)
- `train_labels` এর shape: (60000,)
- `test_images` এর shape: (10000, 28, 28)
- `test_labels` এর shape: (10000,)

অর্থাৎ প্রতিটি ইমেজ একটি **28×28** ম্যাট্রিক্স, এবং প্রতিটি লেবেল একটি সংখ্যা (0–9), যা একটি নির্দিষ্ট পোশাকের ক্যাটেগরি নির্দেশ করে।

3. পদ্ধতি

এই অংশে আমরা সম্পূর্ণ কাজটি ধাপে ধাপে সম্পন্ন করেছি। প্রতিটি ধাপে ব্যবহৃত পদ্ধতি, উদ্দেশ্য এবং কার্যকারিতা বিস্তারিতভাবে ব্যাখ্যা করা হলো। এছাড়াও, এই অংশে আমরা দেখিয়েছি কীভাবে ব্যাকপ্রোপাগেশন ও অপটিমাইজেশনের মাধ্যমে মডেল ধীরে ধীরে নিজের ভুল কমিয়ে আনে এবং আরও উন্নত পূর্বাভাস দিতে শেখে।

ধাপ ১: লাইব্রেরি ইমপোর্ট ও ডেটাসেট লোডিং যাচাই

প্রথমে আমরা Keras লাইব্রেরি ইমপোর্ট করে Fashion MNIST ডেটাসেটটি লোড করার জন্য প্রস্তুতি নিয়েছি:

```
import keras
```

keras.datasets.fashion_mnist.load_data()

এই ধাপটি মূলত ডেটাসেটটি সঠিকভাবে ডাউনলোড এবং অ্যাক্সেসযোগ্য কিনা তা নিশ্চিত করার জন্য ব্যবহৃত হয়েছে।

ধাপ ২: ডেটাসেট লোড ও সংরক্ষণ

এরপর আমরা ডেটাসেটটিকে ভেরিয়েবলে সংরক্ষণ করেছি, যাতে পরবর্তী ধাপগুলোতে ব্যবহার করা যায়:

```
from keras.datasets import fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

এখানে ট্রেনিং এবং টেস্ট ডেটা আলাদা করা হয়েছে, যা মডেল ট্রেনিং এবং মূল্যায়নের জন্য প্রয়োজনীয়।

ধাপ ৩: ডেটা গঠন বিশ্লেষণ

এই ধাপে আমরা ডেটাসেটের গঠন, বৈশিষ্ট্য এবং বন্টন (distribution) ভালোভাবে বোঝার জন্য বিভিন্ন বিশ্লেষণ করেছি। মডেল তৈরি করার আগে এই ধাপটি অত্যন্ত গুরুত্বপূর্ণ, কারণ এর মাধ্যমে আমরা ডেটার প্রকৃতি সম্পর্কে পরিষ্কার ধারণা পাই এবং পরবর্তী ধাপগুলোর জন্য সঠিক সিদ্ধান্ত নিতে পারি।

৩.১. ডেটার **shape** পরীক্ষা করা

প্রথমে আমরা ট্রেনিং এবং টেস্ট ডেটার shape পরীক্ষা করেছি। এর মাধ্যমে আমরা জানতে পেরেছি ডেটাসেটে মোট কতটি ইমেজ রয়েছে এবং প্রতিটি ইমেজের গঠন কেমন।

- ট্রেনিং ডেটা: 60,000টি ইমেজ
- টেস্ট ডেটা: 10,000টি ইমেজ
- প্রতিটি ইমেজের সাইজ: 28×28 পিক্সেল

এই তথ্য থেকে বোঝা যায় যে প্রতিটি ইমেজ একটি 2D ম্যাট্রিক্স আকারে সংরক্ষিত আছে। এছাড়াও, প্রতিটি ইমেজের জন্য একটি করে লেবেল রয়েছে, যা 0 থেকে 9 পর্যন্ত একটি নির্দিষ্ট ক্যাটেগরি নির্দেশ করে।

এই ধাপটি গুরুত্বপূর্ণ কারণ এটি নিশ্চিত করে যে ডেটাসেটটি সঠিকভাবে লোড হয়েছে এবং মডেলে ব্যবহারের জন্য প্রস্তুত।

```
print("Training data")
print(f"- X = {train_images.shape}, y = {train_labels.shape}")
print(f"- Holds {train_images.shape[0] / 70000 * 100:.2f}% of the overall data")

print("\n")

print("Testing data")
print(f"- X = {test_images.shape}, y = {test_labels.shape}")
print(f"- Holds {test_images.shape[0] / 70000 * 100:.2f}% of the overall data")
```

```
Training data
- X = (60000, 28, 28), y = (60000,)
- Holds 85.71% of the overall data


Testing data
- X = (10000, 28, 28), y = (10000,)
- Holds 14.29% of the overall data
```

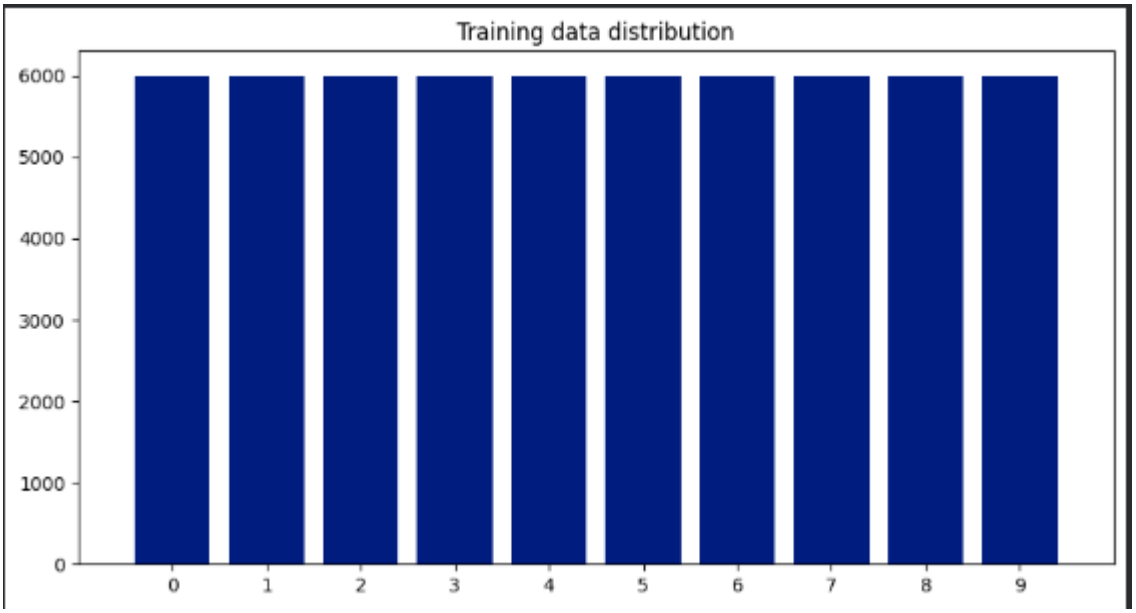
এরপর আমরা ডেটাসেট থেকে কিছু র‍্যান্ডম ইমেজ নির্বাচন করে ভিজুয়ালাইজ করেছি। এর মাধ্যমে আমরা সরাসরি দেখতে পেয়েছি যে ইমেজগুলো কেমন এবং বিভিন্ন ক্লাসের মধ্যে পার্থক্য কী।

- ইমেজগুলো গ্রেস্কেল (black & white)
- প্রতিটি ইমেজে একটি নির্দিষ্ট পোশাকের আকৃতি রয়েছে
- কিছু ক্লাস (যেমন shirt, t-shirt) দেখতে অনেকটা একই রকম

এই ধাপটি আমাদেরকে ডেটার বাস্তব চিত্র (visual understanding) দেয়, যা শুধুমাত্র সংখ্যাগত ডেটা দেখে বোঝা সম্ভব নয়।

৩.৩. লেবেল ডিস্ট্রিবিউশন পর্যবেক্ষণ করা

এই ধাপে আমরা প্রতিটি ক্লাস কতবার উপস্থিত হয়েছে তা বিশ্লেষণ করেছি। অর্থাৎ, কোন ক্লাসের কতগুলো ইমেজ আছে তা একটি বার চার্টের মাধ্যমে দেখেছি।



এই বিশ্লেষণ থেকে আমরা লক্ষ্য করেছি:

- সবগুলো 10টি ক্লাস প্রায় সমান সংখ্যক ডেটা দ্বারা প্রতিনিধিত্ব করছে
- ডেটাসেটে কোনো বড় ধরনের imbalance নেই

এটি একটি গুরুত্বপূর্ণ বিষয়, কারণ যদি কোনো ক্লাস খুব বেশি বা খুব কম থাকত, তাহলে মডেল bias হতে পারত এবং নির্দিষ্ট কিছু ক্লাসের প্রতি বেশি গুরুত্ব দিত।

ধাপ ৪: ডেটা প্রিপ্রসেসিং

মডেলে ইনপুট দেওয়ার আগে raw ডেটাকে উপযুক্ত ফরম্যাটে রূপান্তর করা অত্যন্ত গুরুত্বপূর্ণ। কারণ নিউরাল নেটওয়ার্ক সরাসরি কাঁচা (raw) ডেটা থেকে কার্যকরভাবে শিখতে পারে না। তাই এই ধাপে আমরা কয়েকটি গুরুত্বপূর্ণ প্রিপ্রসেসিং কৌশল প্রয়োগ করেছি, যা মডেলের শেখার ক্ষমতা এবং পারফরম্যান্স উন্নত করে।

৪.১. **Normalization** (নরমালাইজেশন)

প্রথমে আমরা ইমেজের পিক্সেল ভ্যালুগুলোকে 0–255 থেকে 0–1 স্কেলে রূপান্তর করেছি।

```
train_images = train_images.astype('float32') / 255

test_images = test_images.astype('float32') / 255
```

প্রতিটি পিক্সেল মূলত একটি সংখ্যা (0 থেকে 255 পর্যন্ত), যা brightness বা intensity নির্দেশ করে। কিন্তু এই বড় রেঞ্জের মান ব্যবহার করলে মডেলের ট্রেনিং ধীরগতির এবং অস্থির (unstable) হতে পারে।

এই সমস্যা সমাধানের জন্য আমরা প্রতিটি পিক্সেলকে 255 দিয়ে ভাগ করেছি:

$$\text{Normalized value} = \frac{\text{pixel value}}{255}$$

এর ফলে:

- সব পিক্সেল ভ্যালু 0 থেকে 1 এর মধ্যে চলে আসে
- মডেলের training দ্রুত এবং স্থিতিশীল হয়
- gradient descent সহজভাবে কাজ করতে পারে

এই ধাপটি নিউরাল নেটওয়ার্কের জন্য অত্যন্ত গুরুত্বপূর্ণ।

৪.২. **Flattening (**ইমেজকে ভেক্টরে রূপান্তর**)**

ডেটাসেটে প্রতিটি ইমেজের আকার 28×28, অর্থাৎ এটি একটি 2D ম্যাট্রিক্স। কিন্তু আমরা যে মডেলটি ব্যবহার করেছি (Fully Connected Neural Network), সেটি সরাসরি 2D ডেটা গ্রহণ করতে পারে না।

এই কারণে আমরা প্রতিটি ইমেজকে flatten করে 1D ভেক্টরে রূপান্তর করেছি।

```
train_images = train_images.reshape((60000, 784))
test_images = test_images.reshape((10000, 784))
```

28×28=784

অর্থাৎ, প্রতিটি ইমেজ এখন একটি 784 দৈর্ঘ্যের ভেক্টর হিসেবে উপস্থাপিত হয়েছে।

এই ধাপের মাধ্যমে:

- ডেটা মডেলের ইনপুট ফরম্যাটের সাথে সামঞ্জস্যপূর্ণ হয়
- প্রতিটি পিক্সেল একটি ফিচার হিসেবে কাজ করে
- মডেল সহজে ডেটা প্রক্রিয়া করতে পারে

৪.৩. **One-hot Encoding (**লেবেল রূপান্তর**)**

মূল লেবেলগুলো ছিল 0–9 পর্যন্ত সংখ্যা। কিন্তু সরাসরি এই সংখ্যা ব্যবহার করলে মডেল ভুলভাবে ক্লাসগুলোর মধ্যে সম্পর্ক ধরে নিতে পারে।

এই কারণে আমরা one-hot encoding ব্যবহার করেছি:

```
from keras.utils import to_categorical

train_labels = to_categorical(train_labels)

test_labels = to_categorical(test_labels)
```

উদাহরণ:

- যদি লেবেল = 3 হয়, তাহলে এটি রূপান্তরিত হবে:
  [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

এই পদ্ধতিতে:

- প্রতিটি ক্লাস আলাদা ভাবে উপস্থাপিত হয়
- কোনো সংখ্যাগত hierarchy থাকে না
- multi-class classification আরও নির্ভুল হয়

ধাপ ৫: নিউরাল নেটওয়ার্ক আর্কিটেকচার তৈরি

এই ধাপে আমরা একটি **Fully Connected Neural Network (Multilayer Perceptron - MLP)** তৈরি করেছি, যা ইনপুট ডেটা থেকে বৈশিষ্ট্য (features) শিখে সঠিক ক্লাস নির্ধারণ করতে সক্ষম।

মডেলের স্তরগুলো নিম্নরূপ:

- **Input Layer:** 784 ইউনিট (প্রতিটি ইমেজের 784টি পিক্সেল)
- **Hidden Layer:** 512 ইউনিট, ReLU activation সহ
- **Output Layer:** 10 ইউনিট, Softmax activation সহ

```
network = models.Sequential()

network.add(Dense(784, activation='relu', input_shape=(784,)))

network.add(Dense(512, activation='relu'))

network.add(Dense(10, activation='softmax'))
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 784)	615,440
dense_1 (Dense)	(None, 512)	401,920
dense_2 (Dense)	(None, 10)	5,130

Total params: 1,022,490 (3.90 MB)
Trainable params: 1,022,490 (3.90 MB)
Non-trainable params: 0 (0.00 B)

এই আর্কিটেকচারে:

- Input layer প্রতিটি পিক্সেলকে ইনপুট হিসেবে গ্রহণ করে
- Hidden layer ডেটা থেকে গুরুত্বপূর্ণ বৈশিষ্ট্য (patterns) শিখে
- ReLU activation non-linearity প্রদান করে, যা মডেলকে জটিল সম্পর্ক শিখতে সাহায্য করে
- Output layer Softmax ব্যবহার করে প্রতিটি ক্লাসের জন্য probability নির্ধারণ করে

এইভাবে, মডেলটি ধাপে ধাপে ইনপুট ডেটা বিশ্লেষণ করে একটি নির্দিষ্ট ক্লাসে শ্রেণীবদ্ধ করে।

ধাপ ৬: ব্যাকপ্রোপাগেশন (Backpropagation) ও মডেল কম্পাইল

এই ধাপে আমরা মডেলটিকে কম্পাইল করেছি, যা নিউরাল নেটওয়ার্কের শেখার প্রক্রিয়া শুরু করার জন্য অত্যন্ত গুরুত্বপূর্ণ। কম্পাইল করার সময় আমরা নির্ধারণ করি মডেল কীভাবে শিখবে, কীভাবে ভুল নির্ণয় করবে এবং কীভাবে সেই ভুল সংশোধন করবে।

```
network.compile(
  optimizer='rmsprop',
  loss='categorical_crossentropy',
  metrics=['accuracy']
)
```

এখানে তিনটি গুরুত্বপূর্ণ উপাদান ব্যবহৃত হয়েছে:

- **Optimizer (rmsprop):** weights আপডেট করার পদ্ধতি নির্ধারণ করে
- **Loss function (categorical_crossentropy):** মডেলের ভুল (error) পরিমাপ করে
- **Metrics (accuracy):** মডেলের পারফরম্যান্স মূল্যায়ন করে

কীভাবে Backpropagation কাজ করে:

মডেল কম্পাইল করার মাধ্যমে আমরা মূলত এমন একটি সিস্টেম তৈরি করি, যেখানে training এর সময় নিচের চারটি ধাপ বারবার সম্পন্ন হয়:

৬.১. Forward Propagation (সামনের দিকে তথ্য প্রবাহ)

এই ধাপে ইনপুট ডেটা (image) নিউরাল নেটওয়ার্কের প্রতিটি লেয়ারের মধ্য দিয়ে প্রবাহিত হয়।

- প্রতিটি লেয়ারে input → weighted sum → activation function (ReLU/Softmax) প্রয়োগ হয়
- শেষ পর্যন্ত output layer থেকে একটি probability distribution পাওয়া যায়

উদাহরণ:

[image] → hidden layer → output probabilities

এখানে মডেল একটি পূর্বাভাস (prediction) তৈরি করে।

৬.২. **Loss Calculation (**ভুল নির্ণয়**)**

এরপর মডেলের prediction এবং আসল লেবেলের মধ্যে পার্থক্য নির্ণয় করা হয়।

আমরা ব্যবহার করেছি:

- **Categorical Crossentropy Loss**

এই loss functionটি মাপ করে:
মডেলের prediction কতটা আসল লেবেল থেকে দূরে

যদি prediction সঠিক হয় → loss কম
যদি ভুল হয় → loss বেশি

৬.৩. **Backpropagation (**ভুলের বিপরীতমুখী প্রবাহ**)**

এটি এই পুরো প্রক্রিয়ার সবচেয়ে গুরুত্বপূর্ণ অংশ।

- Loss থেকে error নির্ণয় করার পর
- সেই error output layer থেকে শুরু করে ধাপে ধাপে আগের layer-গুলোর দিকে ফিরে যায়

এই সময়:

- প্রতিটি weight এর জন্য gradient (∂Loss/∂Weight) হিসাব করা হয়
- এই gradient বলে দেয় কোন weight কতটা পরিবর্তন করা উচিত

৬.৪. **Weight Update (**ওজন আপডেট**)**

শেষ ধাপে optimizer (RMSprop) ব্যবহার করে weights আপডেট করা হয়।

- Gradient ব্যবহার করে weights একটু করে পরিবর্তন করা হয়
- লক্ষ্য থাকে loss কমানো

RMSprop বিশেষভাবে:

- learning rate নিয়ন্ত্রণ করে
- দ্রুত এবং স্থিতিশীলভাবে convergence নিশ্চিত করে

এখানে উল্লেখযোগ্য যে,আমরা কোডে আলাদাভাবে backpropagation লিখিনি; বরং Keras এর মাধ্যমে এটি স্বয়ংক্রিয়ভাবে সম্পন্ন হয়েছে। বিশেষ করে network.fit() ফাংশনের ভিতরে এই পুরো প্রক্রিয়াটি সম্পাদিত হয়।

অর্থাৎ, যখন আমরা মডেল ট্রেন করি:

- forward pass
- loss calculation
- backward pass (backpropagation)
- weight update

এই সবকিছু একসাথে বারবার ঘটে যতক্ষণ না মডেল ভালোভাবে শিখে।

ধাপ ৭: মডেল ট্রেনিং

এই ধাপে আমরা মডেলটিকে ট্রেন করেছি, যাতে এটি ডেটা থেকে শিখতে পারে:

```
history = network.fit(
  train_images,
  train_labels,
  validation_split=0.2,
  epochs=15,
  batch_size=256
)
```

এখানে:

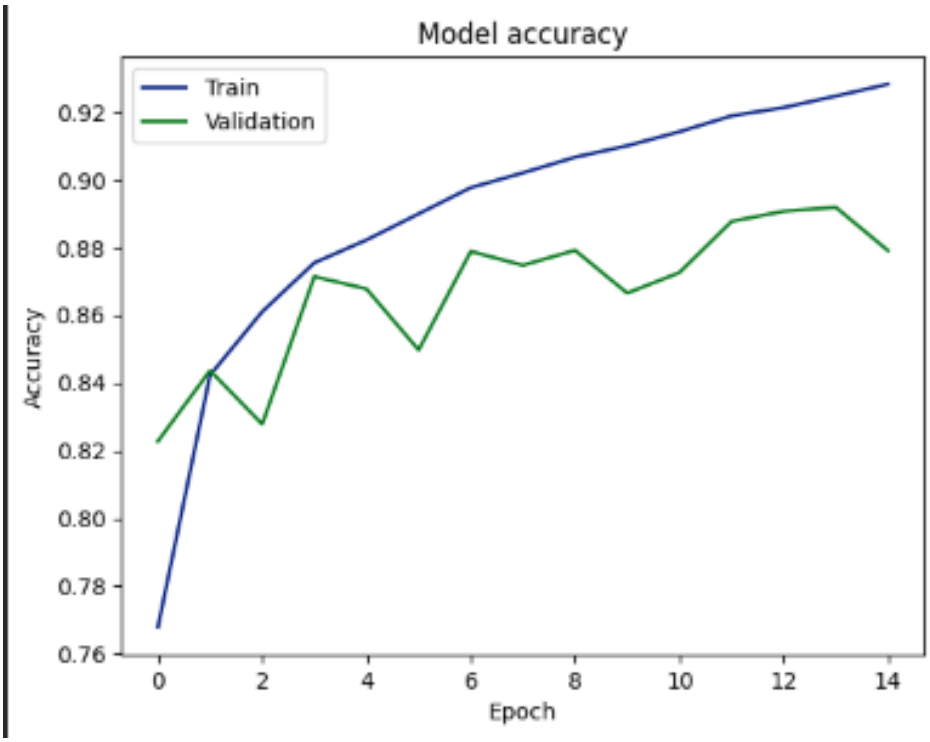
- **Epochs = 15:** মডেল সম্পূর্ণ ডেটাসেট 15 বার দেখে শিখেছে
- **Batch size = 256:** প্রতি ধাপে 256টি ডেটা ব্যবহার করে weights আপডেট করা হয়েছে
- **Validation split = 0.2:** ট্রেনিং ডেটার 20% validation এর জন্য রাখা হয়েছে

```
Epoch 1/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.7679 - loss: 0.6425 - val_accuracy: 0.8227 - val_loss: 0.4830
Epoch 2/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.8423 - loss: 0.4243 - val_accuracy: 0.8436 - val_loss: 0.4109
Epoch 3/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.8610 - loss: 0.3695 - val_accuracy: 0.8278 - val_loss: 0.4429
Epoch 4/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - accuracy: 0.8754 - loss: 0.3350 - val_accuracy: 0.8713 - val_loss: 0.3473
Epoch 5/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 6s 30ms/step - accuracy: 0.8821 - loss: 0.3112 - val_accuracy: 0.8677 - val_loss: 0.3590
Epoch 6/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.8898 - loss: 0.2930 - val_accuracy: 0.8497 - val_loss: 0.4179
Epoch 7/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.8976 - loss: 0.2732 - val_accuracy: 0.8788 - val_loss: 0.3470
Epoch 8/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9019 - loss: 0.2609 - val_accuracy: 0.8747 - val_loss: 0.3438
Epoch 9/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - accuracy: 0.9065 - loss: 0.2471 - val_accuracy: 0.8791 - val_loss: 0.3312
Epoch 10/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - accuracy: 0.9099 - loss: 0.2359 - val_accuracy: 0.8664 - val_loss: 0.3607
Epoch 11/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9140 - loss: 0.2243 - val_accuracy: 0.8725 - val_loss: 0.3840
Epoch 12/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.9187 - loss: 0.2145 - val_accuracy: 0.8876 - val_loss: 0.3202
Epoch 13/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.9212 - loss: 0.2048 - val_accuracy: 0.8906 - val_loss: 0.3164
Epoch 14/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - accuracy: 0.9246 - loss: 0.1959 - val_accuracy: 0.8917 - val_loss: 0.3217
Epoch 15/15
188/188 ━━━━━━━━━━━━━━━━━━━━ 5s 29ms/step - accuracy: 0.9281 - loss: 0.1869 - val_accuracy: 0.8790 - val_loss: 0.3823
```

এই ধাপে মডেল বারবার নিজের পূর্বাভাসের ভুল (error) বিশ্লেষণ করে এবং ব্যাকপ্রোপাগেশনের মাধ্যমে weights আপডেট করে ধীরে ধীরে আরও নির্ভুল হয়ে ওঠে।

৭.১. ট্রেনিং পারফরম্যান্স বিশ্লেষণ

মডেল ট্রেনিং চলাকালীন আমরা accuracy এর পরিবর্তন পর্যবেক্ষণ করেছি। নিচের গ্রাফে ট্রেনিং এবং ভ্যালিডেশন accuracy প্রতিটি epoch এর জন্য প্রদর্শিত হয়েছে:



উপরের গ্রাফটি থেকে আমরা দেখতে পাই যে ট্রেনিং এবং ভ্যালিডেশন উভয় ক্ষেত্রেই মডেলের accuracy ধীরে ধীরে বৃদ্ধি পেয়েছে, যা নির্দেশ করে যে মডেল সফলভাবে ডেটা থেকে শিখছে। প্রথম দিকে ট্রেনিং accuracy দ্রুত বৃদ্ধি পেয়েছে (প্রায় 0.76 থেকে 0.88 পর্যন্ত), যা বোঝায় যে মডেল শুরুতেই গুরুত্বপূর্ণ প্যাটার্নগুলো ধরতে সক্ষম হয়েছে। এরপর ধীরে ধীরে accuracy বৃদ্ধি পেয়ে প্রায় 0.93 এর কাছাকাছি পৌঁছেছে। অন্যদিকে, ভ্যালিডেশন accuracy কিছুটা ওঠানামা (fluctuation) করলেও সামগ্রিকভাবে বৃদ্ধি পেয়েছে এবং প্রায় 0.88–0.89 এর মধ্যে স্থিতিশীল হয়েছে। এটি নির্দেশ করে যে মডেল নতুন ডেটার উপরও ভালো পারফরম্যান্স দেখাচ্ছে।

তবে ট্রেনিং accuracy এবং ভ্যালিডেশন accuracy এর মধ্যে কিছুটা পার্থক্য লক্ষ্য করা যায়, যা হালকা overfitting এর ইঙ্গিত দিতে পারে। অর্থাৎ, মডেল ট্রেনিং ডেটায় কিছুটা বেশি ভালো শিখছে তুলনায় ভ্যালিডেশন ডেটার।

সামগ্রিকভাবে, বলা যায় যে মডেলটি ভালোভাবে ট্রেন হয়েছে এবং গ্রহণযোগ্য মাত্রার accuracy অর্জন করেছে, যদিও আরও উন্নতির জন্য regularization বা tuning করা যেতে পারে।

ধাপ ৮: মডেল মূল্যায়ন (Evaluation)

ট্রেনিং সম্পন্ন হওয়ার পর শুধুমাত্র ট্রেনিং accuracy দেখেই মডেলের পারফরম্যান্স বিচার করা যথেষ্ট নয়। কারণ মডেল ট্রেনিং ডেটা ভালোভাবে শিখলেও নতুন (unseen) ডেটার উপর ভালো কাজ নাও করতে পারে। তাই এই ধাপে আমরা টেস্ট ডেটার উপর মডেলের কার্যকারিতা মূল্যায়ন করেছি।

```
loss, acc = network.evaluate(test_images, test_labels)
```

এখানে:

- **Loss:** মডেলের prediction এবং আসল লেবেলের মধ্যে গড় ত্রুটি নির্দেশ করে
- **Accuracy (acc):** মডেল কত শতাংশ ক্ষেত্রে সঠিকভাবে পূর্বাভাস দিতে পেরেছে তা নির্দেশ করে

এই ধাপের মাধ্যমে আমরা মডেলের **generalization ability** পরীক্ষা করি, অর্থাৎ মডেলটি আগে কখনো না দেখা ডেটার উপর কতটা ভালো কাজ করতে পারে।

যদি টেস্ট accuracy ট্রেনিং accuracy এর কাছাকাছি হয়, তাহলে বোঝা যায়:

- মডেল ভালোভাবে শিখেছে
- overfitting কম হয়েছে

এই ক্ষেত্রে, মডেল প্রায় **87% accuracy** অর্জন করেছে, যা নির্দেশ করে যে মডেলটি গ্রহণযোগ্যভাবে কাজ করছে, যদিও আরও উন্নতির সুযোগ রয়েছে।

```
loss, acc = network.evaluate(test_images, test_labels, batch_size=batch_size)
print("\nTest accuracy: %.1f%%" % (100.0 * acc))
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8745 - loss: 0.4028
Test accuracy: 87.4%
```

এই ধাপটি বাস্তব জীবনে মডেল ব্যবহারের আগে তার কার্যকারিতা যাচাই করার জন্য অত্যন্ত গুরুত্বপূর্ণ।

ধাপ ৯: প্রেডিকশন ও ফলাফল ভিজুয়ালাইজেশন

এই ধাপে আমরা মডেলের আউটপুট আরও গভীরভাবে বিশ্লেষণ করেছি কিছু নির্দিষ্ট টেস্ট ইমেজ ব্যবহার করে। এর মাধ্যমে আমরা শুধু accuracy নয়, বরং মডেল কীভাবে সিদ্ধান্ত নিচ্ছে তা বোঝার চেষ্টা করেছি।

৯.১একক ইমেজে প্রেডিকশন বিশ্লেষণ

প্রথমে আমরা একটি র‍্যান্ডম টেস্ট ইমেজ নির্বাচন করে সেটির উপর মডেলের প্রেডিকশন পরীক্ষা করেছি।

এই অংশের মাধ্যমে আমরা বুঝতে পারি:যে মডেলটি একটি ইমেজকে **"Shirt"** হিসেবে সনাক্ত করেছে, এবং আসল লেবেলও **"Shirt"** ছিল। অর্থাৎ, এই ক্ষেত্রে মডেলটি সঠিকভাবে প্রেডিকশন দিতে সক্ষম হয়েছে।

এছাড়াও, নিচে প্রদর্শিত probability গুলো থেকে বোঝা যায় যে মডেলটি **"Shirt"** ক্লাসের জন্য সর্বোচ্চ **confidence (**প্রায় **0.8198** বা **81.98%)** দেখিয়েছে। অন্যদিকে, "T-shirt/top" ক্লাসের জন্য কিছুটা probability (0.1802 বা 18.02%) থাকলেও অন্যান্য ক্লাসগুলোর probability প্রায় শূন্য।

এটি নির্দেশ করে যে:

- মডেলটি প্রধানত "Shirt" এবং "T-shirt/top" এর মধ্যে সিদ্ধান্ত নিচ্ছিল
- তবে শেষ পর্যন্ত "Shirt" কে বেশি সম্ভাব্য মনে করেছে
- এবং সেটিই সঠিক ছিল

এই বিশ্লেষণ থেকে বোঝা যায় যে মডেলটি সঠিকভাবে কাজ করছে, তবে কাছাকাছি বৈশিষ্ট্যের ক্লাসগুলোর মধ্যে কিছুটা বিভ্রান্তি থাকতে পারে।

৯.২.একাধিক ইমেজে প্রেডিকশন বিশ্লেষণ

এই বিশ্লেষণে আমরা তিনটি বিষয় পর্যবেক্ষণ করেছি:

- **Predicted label (P):** মডেল যে ক্লাসটি অনুমান করেছে
- **True label (T):** আসল বা সঠিক লেবেল
- **Confidence (%):** মডেল কতটা নিশ্চিত সেই প্রেডিকশন সম্পর্কে (Softmax probability)

উদাহরণস্বরূপ:

- যদি P = Shirt এবং T = Shirt → সঠিক প্রেডিকশন
- যদি P ≠ T → ভুল প্রেডিকশন

Confidence (%) আমাদের দেখায়:

- মডেল কতটা নিশ্চিত ছিল তার সিদ্ধান্ত নিয়ে
- এটি Softmax output থেকে আসে (0 থেকে 1 → percentage)

এই ধাপ থেকে যা বোঝা যায়-

এই বিশ্লেষণের মাধ্যমে আমরা কয়েকটি গুরুত্বপূর্ণ বিষয় লক্ষ্য করেছি:

1. সঠিক প্রেডিকশন**:**
   মডেল বেশিরভাগ ক্ষেত্রে সঠিকভাবে ইমেজ শ্রেণীবদ্ধ করতে সক্ষম হয়েছে, বিশেষ করে পরিষ্কার এবং স্পষ্ট ইমেজগুলোর ক্ষেত্রে।

2. ভুল প্রেডিকশন**:**
   কিছু ক্ষেত্রে মডেল ভুল করেছে, বিশেষ করে যখন দুইটি ক্লাসের বৈশিষ্ট্য কাছাকাছি (যেমন: sneaker vs sandal, shirt vs t-shirt)।
   এটি নির্দেশ করে যে মডেল কখনো কখনো সূক্ষ্ম পার্থক্য বুঝতে ব্যর্থ হয়।

3. উচ্চ **confidence** সহ ভুল**:**
   কিছু ভুল প্রেডিকশনে মডেল উচ্চ confidence (যেমন 99%) দেখিয়েছে।
   এটি একটি গুরুত্বপূর্ণ সীমাবদ্ধতা, কারণ মডেল ভুল হলেও নিজেকে খুব নিশ্চিত ভাবছে।

বলা যায়, এই ধাপের মাধ্যমে মডেলের প্রেডিকশন ক্ষমতা, সিদ্ধান্ত গ্রহণ প্রক্রিয়া এবং সীমাবদ্ধতা সম্পর্কে স্পষ্ট ধারণা পাওয়া গেছে, যা ভবিষ্যতে মডেল উন্নয়নের জন্য গুরুত্বপূর্ণ দিকনির্দেশনা প্রদান করে।

4.উপসংহার

এই ল্যাব রিপোর্টে আমরা Fashion MNIST ডেটাসেট ব্যবহার করে একটি নিউরাল নেটওয়ার্ক মডেল তৈরি এবং বিশ্লেষণ করেছি। শুরুতে ডেটাসেট লোড করা, ডেটা বিশ্লেষণ এবং প্রিপ্রসেসিংয়ের মাধ্যমে ডেটাকে মডেলের উপযোগী করে তোলা হয়েছে। এরপর একটি Fully Connected Neural Network তৈরি করে সেটিকে ট্রেনিং করানো হয়েছে, যেখানে ব্যাকপ্রোপাগেশন অ্যালগরিদম ব্যবহার করে মডেল ধীরে ধীরে নিজের ভুল কমিয়ে শিখেছে।

মডেল ট্রেনিং শেষে আমরা এর পারফরম্যান্স মূল্যায়ন করেছি এবং দেখতে পেয়েছি যে মডেলটি গ্রহণযোগ্য মাত্রার accuracy অর্জন করেছে। এছাড়াও, একক এবং একাধিক ইমেজের উপর প্রেডিকশন বিশ্লেষণের মাধ্যমে মডেলের বাস্তব আচরণ, সিদ্ধান্ত গ্রহণ প্রক্রিয়া এবং সীমাবদ্ধতা সম্পর্কে ধারণা পাওয়া গেছে।

সামগ্রিকভাবে, এই কাজটি নিউরাল নেটওয়ার্ক এবং ব্যাকপ্রোপাগেশন অ্যালগরিদমের বাস্তব প্রয়োগ বুঝতে সাহায্য করেছে। যদিও মডেলটি বেশ ভালো ফলাফল দিয়েছে, তবুও ভবিষ্যতে আরও উন্নত মডেল (যেমন CNN) ব্যবহার করে এবং হাইপারপ্যারামিটার টিউনিংয়ের মাধ্যমে এর পারফরম্যান্স আরও বৃদ্ধি করা সম্ভব।